Group Members:

Valentina Susarret - vsusar3

Sean OReilly - sorei2

Dominic Aidoo - daido2

Youssef Elghawabi - yelgh2

CS362 Project - Programmable Toy Car

GitHub Repository

Abstract

This project is a programmable LEGO toy car that follows a path set by a user. Utilizing an onboard interface, users input up to ten directional moves via buttons and select speeds using a potentiometer. Inside, four Arduinos work together using our own custom JSON communication code. One Arduino acts as the main brain, while the others control the driving, steering, and an LCD screen. The car also has an ultrasonic sensor to avoid crashing into obstacles. Overall, it is an original project that shows how multiple Arduinos can connect and share tasks.

Project Ideas

This project is a programmable toy car that executes a user-defined sequence of up to ten cardinal-direction moves. In “reading mode,” users enter moves using four directional buttons and select one of three speeds (slow, moderate, fast) with a potentiometer. The car switches to “driving mode” when the “GO” button is pressed or the move limit is reached. An onboard ultrasonic sensor provides automatic collision avoidance. The car’s shell and components (e.g., wheels) are built from LEGO pieces.

Updated Arduino Usage

The system uses a master–slave architecture with four Arduinos (AM, A1, A2, A3). The Master (AM) processes input from A3 and sends movement commands to A1 and A2. A1 controls rear-wheel drive (speed, duration, direction), while A2 controls steering. A3 collects button inputs, displays them on an LCD, and forwards them to AM for processing.

Updated Arduino Communication

The Arduinos communicate over serial. We have developed a shared library that allows Arduinos to share code and a common networking interface. This will allow the master arduino to send 4 byte messages and receive responses from the other Arduinos. The functions in the shared library will block execution, allowing the master arduino to properly wait for responses and sync behavior between all Arduinos together. Each receiving Arduino executes specific functions such as driving, steering, or logic control based on the incoming data. All of our message data is encoded in single byte enumerations which fit into small buffers that are suitable for the hardware. There is further information about our communication plan in this document and a working prototype can be found in our GitHub repository.

Updated Inputs and Outputs

Inputs include four directional push buttons, a “GO” button, a potentiometer for speed control, and an ultrasonic sensor for collision detection. Outputs include an LCD displaying the programmed path and selected speed, an LED that lights while the car executes the path, and a buzzer that sounds when an obstacle is detected.

Updated Original Work

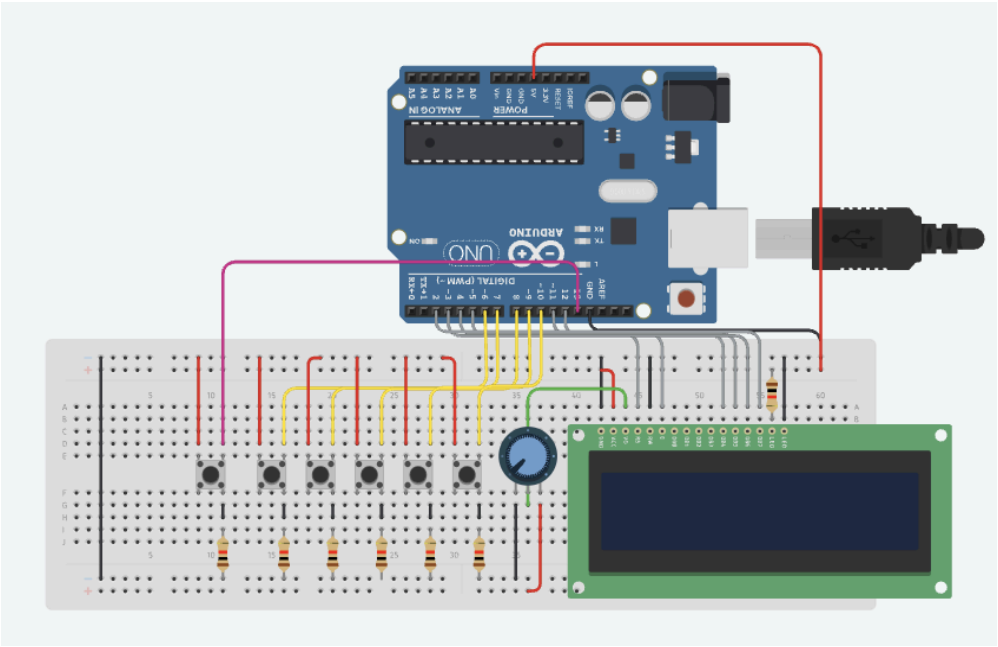
This project comprises original work, as the system architecture and multi-Arduino integration resulted from our group's collaboration rather than copying external sources. Other unique elements such as our own LEGO design for the shell, writing our own serial communication API, the programmable path, and controller module being on device rather than in the form of a remote control proves that our project has original work.

Updated List of Materials

This project uses four Arduinos and eight device types: one stepper motor, two servo motors, five push buttons, one potentiometer, one LCD display, one LED, one buzzer, and one ultrasonic sensor. For power, we will need at least 6xAA batteries in a battery holder. The physical structure of the toy car will be built using LEGO pieces, including wheels, base plates, and beams to form the base and roof. The car's dimensions will depend on Arduino placement and overall layout, with an estimated footprint between 6 in × 6 in and 12 in × 4 in, and a height ranging from 3 to 6 inches.

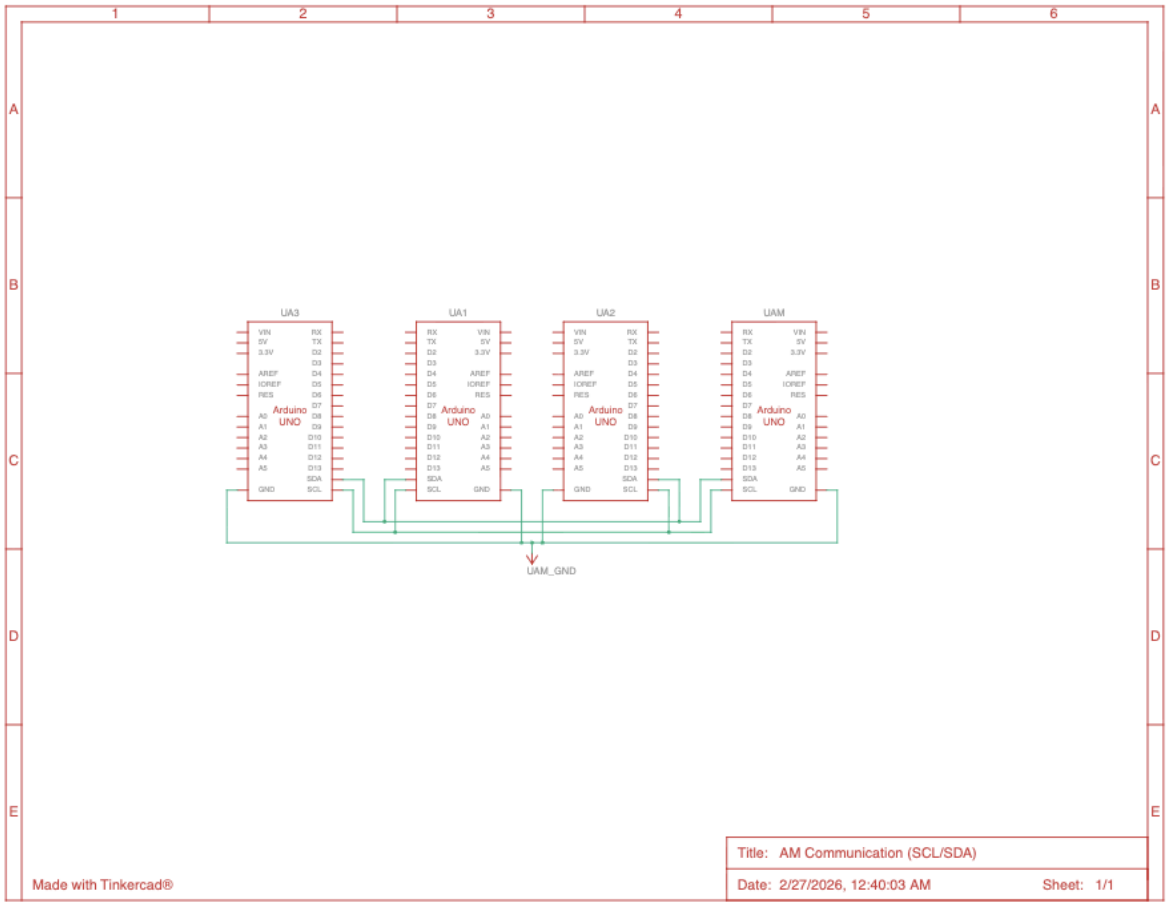
Arduino Diagrams

A3 design



Arduino Schematics

AM connected to A1, A2, and A3 via SCL/SDA.



Updated Code Sketches

ProgCar.h - Nearly complete library enabling blocking send and receive functionality between a master arduino and multiple slave arduinos

```
#include <Arduino.h>
#include <Wire.h>

namespace pgc
{
    constexpr int BAUD_RATE = 9600;

    constexpr int A1_ADDRESS = 8;
    constexpr int A2_ADDRESS = 9;
    constexpr int A3_ADDRESS = 10;

    constexpr int MAX_MOVES = 10;
```

```
constexpr int MESSAGE_SIZE = 4;
byte slaveResponseBuffer[MESSAGE_SIZE];

/*
Protocol send and response codes
Can be bitwise OR'd together
*/
constexpr byte NULL_RESPONSE = 0x00;
constexpr byte RUN_COMMAND = 0x01;
constexpr byte WORK_DONE = 0x02;
constexpr byte NOT_READY = 0x04;

enum class Gas : byte
{
    FORWARD,
    REVERSE
};

enum class SteerDir : byte
{
    LEFT,
    STRAIGHT,
    RIGHT
};

enum class Speed : byte
{
    LOW_SPEED,
    MEDIUM_SPEED,
    HIGH_SPEED
};

struct alignas(4) Move
{
    Gas gas;
    SteerDir steer;
    Speed speed;
};

void reportToMaster(const Move& move)
{
    slaveResponseBuffer[0] = static_cast<byte>(move.gas);
    slaveResponseBuffer[1] = static_cast<byte>(move.steer);
    slaveResponseBuffer[2] = static_cast<byte>(move.speed);
    slaveResponseBuffer[3] = 0; // junk byte
}
```

```
    void reportToMaster(const byte status)
    {
        for (int i = 0; i < MESSAGE_SIZE; ++i)
        {
            slaveResponseBuffer[i] = status;
        }
    }

    namespace // Slave event handlers
    {
        bool slaveResponseReady = false;
        bool slaveCommandShouldRun = false;

        /* Called when master sends slave a command to execute
*/
        void onSlaveCommandRecv(int bytesRecv) {
            if (bytesRecv <= 0)
                return;

            if (!slaveResponseReady) // Only carry out command
if move isn't already ready
            {
                byte command = Wire.read();

                if (command == RUN_COMMAND) {
                    slaveCommandShouldRun = true;
                }
            }
        }

        /* Called when master requests a response from slave */
        void onSlaveResponseRequested() {
            if (slaveResponseReady) {
                Wire.write(slaveResponseBuffer,
sizeof(slaveResponseBuffer));
                slaveResponseReady = false;
            } else {
                byte notReady[MESSAGE_SIZE] = {NOT_READY,
NOT_READY, NOT_READY, NOT_READY};
                Wire.write(notReady, MESSAGE_SIZE);
            }
        }
    }

    /* Will call func if master has requested slave to run */
```

```
    void handleMasterRequest(void (*func)())
    {
        if (slaveCommandShouldRun) {
            slaveCommandShouldRun = false;
            for (int i = 0; i < MESSAGE_SIZE; ++i)
            {
                slaveResponseBuffer[i] = NULL_RESPONSE;
            }
            func();
            slaveResponseReady = true;
        }
    }

    void initAM()
    {
        Wire.begin();
        Serial.begin(BAUD_RATE);
    }

    void initSlave(const int address)
    {
        Wire.begin(address);
        Wire.onReceive(onSlaveCommandRecv);
        Wire.onRequest(onSlaveResponseRequested);
        pinMode(LED_BUILTIN, OUTPUT);
    }
}
```

Code sketch for steering arduino(A2):

```
#include "ProgCar.h"

#include <Servo.h>



using namespace pgc;
```

```
//  Hardware Setup
Servo steeringServo;
const int SERVO_PIN = 3;
// Steering Angles
const int ANGLE_LEFT = 45;
const int ANGLE_STRAIGHT = 90;
const int ANGLE_RIGHT = 135;


//  Execution Function
void executeSteering() {
  SteerDir currentDir = SteerDir::STRAIGHT;


  // Read the payload sent by the Master
  if (Wire.available()) {
    currentDir = static_cast<SteerDir>(Wire.read());
  }


  // Command the servo (this triggers the hardware timer, no blocking required)
  switch (currentDir) {
    case SteerDir::LEFT:
      steeringServo.write(ANGLE_LEFT);
      Serial.println("Action: Commanded LEFT");
      break;
```

```
    case SteerDir::RIGHT:
      steeringServo.write(ANGLE_RIGHT);
      Serial.println("Action: Commanded RIGHT");
      break;
    case SteerDir::STRAIGHT:
    default:
      steeringServo.write(ANGLE_STRAIGHT);
      Serial.println("Action: Commanded STRAIGHT");
      break;
  }

  // Immediately report success.
  // The servo will physically travel to its position in the background
  // while the Master is freed up to immediately trigger the Drive Arduino (A1).
  reportToMaster(WORK_DONE);
}

void setup() {
  Serial.begin(BAUD_RATE);

  // Initialize this Arduino as Slave A2
  initSlave(A2_ADDRESS);
```

```
  // Attach the servo and set it to straight by default

  steeringServo.attach(SERVO_PIN);

  steeringServo.write(ANGLE_STRAIGHT);



  Serial.println("A2 Steering Arduino Initialized (Non-Blocking).");

}



void loop() {

  // Checks for commands and runs executeSteering instantly when triggered

  runSlaveCommand(executeSteering);

}
```

Code Sketch for Rear Drive Arduino (A1):

```
#include <Arduino.h>
#include <Wire.h>
#include <ProgCar.h>

// Stepper motor control pins
const int IN1 = 2;
const int IN2 = 3;
const int IN3 = 4;
const int IN4 = 5;
```

```
// I2C address for this slave
#define A1_ADDRESS 0x01

// Control variables
Gas currentGas = Gas::FORWARD;

// Step sequence (half-step or full-step simplified)
int stepIndex = 0;

int stepSequence[4][4] = {
  {1, 0, 0, 1},
  {1, 0, 0, 0},
  {1, 1, 0, 0},
  {0, 1, 0, 0}
};

// Apply step to motor pins
void setStep(int a, int b, int c, int d) {
  digitalWrite(IN1, a);
  digitalWrite(IN2, b);
  digitalWrite(IN3, c);
  digitalWrite(IN4, d);
}

// Drive logic
void executeDrive() {
  // Read command from AM
  if (Wire.available() >= 1) {
    int gasVal = Wire.read();
    currentGas = (Gas)gasVal;
  }

  // Step direction
  if (currentGas == Gas::FORWARD) {
    stepIndex++;
  } else if (currentGas == Gas::BACKWARD) {
    stepIndex--;
  }

  // Wrap step index
  if (stepIndex >= 4) stepIndex = 0;
  if (stepIndex < 0) stepIndex = 3;

  // Apply step sequence
  setStep(
    stepSequence[stepIndex][0],
```

```
    stepSequence[stepIndex][1],
    stepSequence[stepIndex][2],
    stepSequence[stepIndex][3]
  );

  sendStatus(WORK_DONE);
}

void setup() {
  Serial.begin(9600);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  initSlave(A1_ADDRESS);
}

void loop() {
  handleMasterRequest(executeDrive);
}
```

Code sketch of communication between four arduinos:

```
Library.h
// Code shared between all Arduinos

#include <Arduino.h>
#include <ArduinoJson.h>
#include <Wire.h>


// Global variables for readability
AM = // AM's serial address
A1 = // A1's serial address
...

sendMessage(address, message):
   Wire.begin(address)
   Wire.send(message.ToJson())

recvMessage(address) -> returns message:
     if Wire.available()
          message = deserializeJson(readWire())
          return message
   else: return null message

// Message structured data
struct Message
   enum type (MOVE, EXECUTE, DONE...)
   field1
   field2

   ToJson()
       return serializeJson(type, field1...)
 ...

class PushButton
{

...
```

```
AM

#include "Library.h"
#include "Wire.h"

setup():
   // Set serial baud
   // Setup wire

loop():

   sendMessage(A3, message=START)

   // Poll messages from A3
   messageRecv = recvMessage(A3)

   while(messageRecv is null)
       // receive more messages
       messages.append(messageRecv)
       // if message.type == EXECUTE, break

   sendMessages(A1, messages)
   sendMessages(A2, messages)

   // wait for A1 and A2 to execute
   while(messageRecv(A1) is null or not DONE)
   while(messageRecv(A2) is null or not DONE)
```

```
A1 (Rear Wheel Drive)

#include "Library.h"

setup():
  // Setup serial
  // Setup wire

loop():

  // poll messages from AM

   messageRecv = recMessage(AM)

  while(receiving messages)
      messages.appendMessages
      if message.type == EXECUTE, break

      // parse message into cardinal move

      for move in moves
          execute move

   sendMessage(AM, message=DONE)
```

```
A2 (Front Steering)

#include "Library.h"

setup():
  // Setup serial
  // Setup wire

loop():

  // poll messages from AM

   messageRecv = recMessage(AM)

  while(receiving messages)
      messages.appendMessages
      if message.type == EXECUTE, break

      // parse message into cardinal move

      for move in moves
          execute move

   sendMessage(AM, message=DONE)
```

```
A3 (Input / UI)

#include "Library.h"
#include "LiquidCrystal.h"

setup():
 // Setup wire, serial

loop():
   // Start empty input and LCD

   // poll input from user (push buttons, LCD)

    while(input != EXECUTE)
       // update LCD screen & list of messages


    sendMessages(AM, messages)


    while (recvMessage(AM) != DONE)
       // wait for AM to notify A3 move is done
```

Discussion On How To Build The Project

Each Arduino should be grounded together, with serial connections made between the SDA/SCL pins so they can communicate.

AM:

The AM (master Arduino) is in charge of syncronizing actions between all Arduinos together. Its purpose is to get user input data from the input Arduino, then broadcast that data to the driving and steering Arduinos so they can run the appropriate commands. The AM's loop is essentially the high level functionality of the programmable car as a whole. To achieve this, the AM needs its code and the shared library code. The AM will need to be grounded to and wired via SDA/SCL pins to the three other Arduinos so they can communicate via serial. The AM will also need to be powered since it will run independently of a connection to a computer.

A1:

A1 is responsible for controlling the rear wheels. The arduino will receive data from the AM regarding the direction (front/backwards). The A1 Arduino is connected to the Master Arduino using I2C communication, where the A4 (SDA) pin connects to the Master's A4 pin and the A5 (SCL) pin connects to the Master's A5 pin, and both Arduinos share a common GND connection. The stepper motor driver module is powered by a 6xAA battery pack, with the battery positive connected to the driver's motor power input (VMOT/VCC) and the battery negative connected to the driver's GND. The driver's GND is also connected to the Arduino GND to maintain a common reference across the system. The stepper motor driver control pins are connected to Arduino digital pins 2, 3, 4, and 5, which send step signals to control the motor's rotation and direction. The stepper motor is connected to the output terminals of the driver module, which physically drives the rear wheel assembly built using LEGO components.

A2:

Power wire for the servo motor is connected to the 5V rail. Ground wire for the servo motor is connected to the GND rail and the signal wire is connected to digital pin 3 on the arduino. The

A4 pin on the arduino will be connected to the A4 pin on the Master Arduino. The A5 pin will be connected to the A5 pin on the Master Arduino. This arduino will be connected to the master Arduino via GND rail.

A3:

The LCD is wired as such: LCD RS pin to digital pin 12, LCD Enable pin to digital pin 11, LCD D4 pin to digital pin 5, LCD D5 pin to digital pin 4, LCD D6 pin to digital pin 3, LCD D7 pin to digital pin 2, LCD R/W pin to GND, LCD VSS pin to GND, LCD VCC pin to 5V, LCD LED+ to 5V through a 220 ohm resistor, LCD LED- to GND. We also wired a 10k potentiometer to +5V and GND, with its wiper (output) to LCD screens VO pin (pin3). The drive button is connected to pin 13, the reverse button is connected to pin 6, the steer-left button is connected to pin 7, the steer-right button is connected to pin 8, the speed button is connected to pin 9, the execute button is connected to pin 10. The A3 will need to be grounded to and wired via SDA/SCL pins to the AM can communicate via serial.

LEGO Shell:

The shell of the toy car will be built out of the following pieces: 4 12x4 base plates, 4 wheels, and 1cm high walls to keep Arduinos in place. The design is a flat sheet that has each Arduino and its components lined up next to each other to allow for easy reaching and simplicity of building.

How The Project Is To Be Used

Usage of the programmable car should be straightforward. The user will interact with the A3 input Arduino's controls to change the speed, direction, and steering of the car. Once the user wants to run the sequence of directions they entered into the car, they can hit a button to send the car off. The car will execute a sequence of commands using its motors to go forward, backward and steer around, and will stop if it reaches an obstacle. Once the car has finished driving, the user can repeat the process and add in new directions for the car to drive with.

Updated Timeline

COMPLETED: Week 1 (Mon 3/2 – Sun 3/8/26) – Role Assignment & System Setup

- Review and confirm finalized sketches and planned code structure.
- Divide Arduino responsibilities:
    - AM (Master)
    - A1 (Rear Drive)
    - A2 (Steering)
    - A3 (User Interface)
- Get some code sketches in the GitHub repo as pull requests.
- Build a defined set of input behaviors and constraints. (Diagrams, pseudo code) and how each "move" will look for the car. - what does each do exactly?
- Create finalized list of materials needed (devices, etc.)
- Verify all hardware components are available and functional.

COMPLETED: Week 2 (Mon 3/9 – Sun 3/15/26) – Initial Design & Module Setup

- Set up individual Arduino modules (A1, A2, A3, AM) for testing.
- Test communication between Master (AM) and Slaves (A1, A2, A3).
- Prototype push button input and LCD display for path input.
- Verify basic motor and servo functionality.
- **Prep slides and diagrams for Design Presentation.**

COMPLETED: Week 3 (Mon 3/16 – Sun 3/22/26) – Parallel Module Development

Monday 3/16: Deliver Design Presentation

- Incorporate feedback.

Then everyone works on their assigned board:

A3 (User Interface Developer)

- Finalize button input logic
- Implement path storage array (max 10 moves)
- Display live path on LCD
- Implement GO button trigger signal to AM

AM (Master Developer)

- Implement serial receive from A3
- Store received path
- Create drive-mode state machine
- Implement LED control logic
- Begin ultrasonic sensor testing

A1 (Rear Drive Developer)

- Finalize servo motor control functions
- Create movement functions (forward, backward, stop)
- Implement serial receive from AM
- Test timed motion control

A2 (Steering Developer)

- Finalize stepper motor control
- Create turn-left / turn-right functions
- Implement serial receive from AM
- Test steering precision

COMPLETED: Week 4 (Mon 3/23 – Sun 3/29/26) – Communication Integration

- Connect A3 ↔ AM and verify clean data transfer
- Connect AM ↔ A1 and verify movement commands
- Connect AM ↔ A2 and verify steering commands
- Implement full reading mode → driving mode transition
- Start combining steering + drive timing

COMPLETED: Week 5 (Mon 3/30 – Sun 4/5/26) – General Implementation

- Finish implementing A3 hardware and code
- Finalize plan for remaining month.

Week 6 (Mon 4/6 – Sun 4/12/26) – Individual completion of modules

- Valentina and Sean will meet on Monday to connect A3 and AM for testing.
- All members complete their modules.
- Push code to GitHub whenever a change is made.
- Test implementations via TinkerCad and physical Arduino setups.

Week 7 (Mon 4/13 – Sun 4/19/26) – First assembly of project

- Weekly meeting is in person to assemble modules and do a test run
- Plan the weekend to find what must be fixed for Arduinos to operate.

Week 8 (Mon 4/20 – Sun 4/26/26) – Finalize project and demo

- Finalize project build and perform mock demos for testing
- Script what the official live demo will look like, (what path will be followed, obstacles placed sensed)

Week 9 (Mon 4/27 – Wed 4/29/26) – Project Demonstration & Documentation

- **Monday 4/27:** Deliver **Project Demonstration**.
- **Wednesday 4/29:** Complete **Final Design Document** including schematics, code listings, and system overview.

Updated References

[1] BenoitBlanchon, "ArduinoJson: Efficient JSON serialization for embedded C++," *ArduinoJson*. https://arduinojson.org/

[2] "Serial Input Basics - updated," *Arduino Forum*, Apr. 25, 2016. https://forum.arduino.cc/t/serial-input-basics-updated/382007

[3] "Serial," *Arduino.cc*, 2024. https://docs.arduino.cc/language-reference/en/functions/communication/Serial/

[4] *Arduino.cc*, 2024. https://docs.arduino.cc/language-reference/en/functions/communication/wire/

[5] R. Y. ChannelFollow, "Arduino I2C and Multiple Slaves," *Instructables*. https://www.instructables.com/Arduino-I2C-and-Multiple-Slaves/

[6] "Wire", *Arduino.cc*, 2024.
https://docs.arduino.cc/language-reference/en/functions/communication/wire/

[7]Arduino, "Servo Motor Basics with Arduino," *docs.arduino.cc*, Mar. 04, 2024.
https://docs.arduino.cc/learn/electronics/servo-motors/

[8]Arduino, "Arduino and Stepper Motor Configurations | Arduino Documentation," *docs.arduino.cc*, Jan. 25, 2022. https://docs.arduino.cc/learn/electronics/stepper-motors/